

Operating Systems – Lecture & Tutorial Notes

Contents

1	Lecture 2	3
2	Tutorial 1	4
3	Tutorial 2	4
4	Lecture 3	6
5	Tutorial 3	8
6	Tutorial 4	9
7	Lecture 4	10
8	Lecture 5	11
9	Tutorial 5	13
10	Lecture 6	14
11	Tutorial 6	14
12	Lecture 7	15
13	Tutorial 7	15
14	Lecture 8	16
15	Lecture 9	17
16	Tutorial 9	18
17	Tutorial 10	18
18	Lecture 11	19
19	Lecture 12	19

Lecture 2

Processes

1. When a process is waiting, it is sleeping or “blocked”.
2. When we context switch to another process we save the current PCB and load the next PCB.
3. `wait` will block unless `WNOHANG` is given as an option, otherwise it will run in parallel.
4. `fork` returns 0 if the current process is the child, otherwise it returns > 0 (PPID).
5. Theoretically the printing order is not defined. But practically there are locks on printing and accessing the STD I/O.
6. `errno` is a global variable storing the error number of the last syscall. Before checking the error value of the last syscall, `errno` needs to be zeroed.
7. `exit` should not fail.
8. `execv` never returns unless it fails.
9. The init process (PID 1) acts like a parent for all the orphan processes.

Signals

1. When a signal is received, the process stops whatever it is doing and handles it. They cannot be ignored unless that is their handler.
2. The default handler is ignore or die.
3. You cannot ignore the behavior of certain signals: `Ctrl-Z` (stop), `SIGSTOP` (makes the process sleep), `SIGKILL`, `SIGCONT`.
4. You cannot change the behavior of certain signals: `SIGSTOP`, `SIGKILL`. You can change the handler of `SIGCONT`.
5. When the signal handler does not exit, the process resumes where it last was.
6. Ignoring a signal is done using `SIG_IGN`. `SIG_DFL` is the default behavior.
7. `SIGBUS` – invalid dereference. `SIGILL` – illegal instruction. `SIGSEGV` – segmentation fault. `SIGFPE` – arithmetic errors. All of these are HW interrupts. The hardware sends an interrupt, the OS handles it and as part of the handling process it sends a signal to the process in question.
8. Every CPU has a soft limit and a hard limit. `SIGXCPU` is sent to a process if it passes the soft limit.
9. When a process exceeds a hard limit, a `SIGKILL` is sent.
10. `SIGPIPE` – write to pipe with no readers.
11. Processes can fire signals using the system call `kill(pid, sig)`.
12. Signals are invoked when returning from kernel to user mode.

13. Signals being asynchronous may lead to race conditions. That is why they are allowed to be blocked.
14. The PCB keeps a list of all the process signals that have been blocked.
15. **SIGIO** is to be fired whenever some I/O is ready. This only makes sense when we are talking about a non-blocking file descriptor.
16. Signal mask (**sigmask**): the current signals being blocked by the process. When using **sigblock** the set is the union of the old mask and the new mask. When using **sigunblock** it is the set difference of the old mask and the new mask. When using **setmask** it sets the old to the new.
17. If a signal is fired when we are in kernel mode, the syscall fails and returns **EINTR** in **errno**.

Questions

- Do signals cause context switches? When do context switches occur?
- What does a child process not inherit?
- Difference between all the different **wait** functions?
- When does a **wait** block?
- Syscalls that should not fail? **exit**. What else?
- **SIGIO**?
- What does point 12 mean?

Tutorial 1

1. Interrupts are handled in between instructions, such that the execution of an instruction is not interrupted.
2. Interrupts can occur when we are in the kernel too; in other words, interrupts can interrupt syscalls.
3. Can syscalls interrupt interrupts? No. Interrupts are handled in the kernel anyway and we would not need to use syscalls in the kernel code.

Questions

- Where is the exit value of a program returned? Does it contain any info relevant to its waiting parent?

Tutorial 2

1. PID is a 32-bit number. Most OSes use the 15 least significant bits, so about 32k processes. The admin can configure a higher number.
2. Idle has PID 0.

3. Idle creates init, which has PID 1. Init creates the rest of the processes.
4. `printf` is thread safe.
5. `wait(int* wstatus)` – status has info about the status of the terminating child process, i.e. the exit value; it is in the second byte of the 4 bytes. That same value is saved in `WEXITSTATUS`.
6. When using `wait` – if there are no child processes or if they have all finished running, it returns `-1`.
7. `wait` and `waitpid` are blocking syscalls. To avoid this, we can use the flag `WNOHANG` as an argument for `waitpid`; that way, if the child process has not finished running, it will return 0 immediately.
8. `exit` does not return. `exit` is used as a standard part of the termination of a process. `main` is called by the libc function `__libc_start_main()`.
9. The struct of a child process is only really deleted once the parent collects the termination status.
10. In `argv`, `argv[0]` is the name of the executable program and `argv[n-1] = NULL`.
11. If `execv` fails it returns `-1`, otherwise it does not return at all.
12. Non-blocking syscalls: `getpid`, `getppid` – this is very important as it helps us understand that the OS will not put the process to sleep and schedule another one inherently.
13. Parent vs real parent: `parent` is the current parent, whereas the real parent is the parent that created the process. When talking about the Linux fields, if the real parent no longer exists, the field will point to the PCB of init. Real parent and parent may differ sometimes, such as when another process is monitoring the process using the system call `ptrace` (for debugging purposes). The parent is always signaled when a child process terminates (`SIGCHLD`), and the default response is to ignore it.
14. The OS keeps a list of the current processes. The head process is idle.
15. The OS keeps a hash table for all the processes.
16. The state of the program is saved in a 32-bit variable that acts as an array. One bit is on at a time. When a process is in `TASK_INTERRUPTIBLE` state, it can be interrupted via a signal (when it is waiting for a child, for example). When a task is stopped, it has been stopped by another process (tracer or debugger).
17. Difference between runqueue and process list: the runqueue only has the processes that are ready to run (`TASK_RUNNING`).
18. Tasks are added to the runqueue using `activate_task`.
19. In Linux there are waiting queues for all the tasks in the states `TASK_INTERRUPTIBLE` and `TASK_UNINTERRUPTIBLE`. The only way a process is moved there is if it uses a blocking syscall, like `wait`, `read`, `write`, etc.

*“And I, give all moments for the sky, and time, it wasn’t yours it wasn’t mine.
And I wasn’t taken from for nothing. It was always for the sky.”*

Lecture 3

Threads

1. Multiprocessing is when we use multiple CPU cores to run a single process – an example of this is web servers. This is done by forking and letting the child processes handle the actual requests (for now; actually uses threads for the most part). The parent process will only ever receive requests. It is often done using a bound number of pre-forked children because forking on the fly is “expensive”.
2. Forking is a spectrum: on one end exists a forked process and on the other exists a thread.
3. Threads share everything except for the stack and the registers.
4. OpenMP – the code looks like it is linear but when compiled the compiler adds multithreading directives, making the process multithreaded.
5. There is also the `<thread>` library, which is a C++ library.
6. Pthreads is the library most often used.
7. Windows and Solaris implement threads and processes differently, whereas Linux does not.
8. In Linux, `pthread_create` and `fork` are just wrapper functions of `clone`. `clone` is the actual syscall that does the process duplication/cloning. In other words, both `pthread_create` and `fork` use `clone` in their implementation in Linux.
9. In Linux, everything is a file. One way to obtain a file descriptor is by opening it.
10. Pipes allow for sending messages between a child and its parent. A pipe is just a pair of file descriptors. `pipe[0]` is for reading and `pipe[1]` is for writing.
11. Reminder: `read` is a blocking syscall. Therefore when a process tries to read from a pipe that has not been written to yet, it will keep waiting until something finally gets written.
12. Reminder: writing to a pipe with a closed reading end will fire `SIGPIPE` (writing to a pipe with no readers as well).
13. We create pipes using the syscall `pipe`, which takes an array of two `ints` as an argument.
14. When creating a thread we can give the function we would like it to run as an argument. In addition, we can wait for the threads to finish.
15. A parent and its forked child will not share an array, but a parent’s threads will.
16. Threads can be implemented at user level.

Context Switching Overhead

1. If we assume that a context switch takes c time and that a task takes $c \times k$ time, then the rate of cycles the context switch takes is $c/(c \times k + c) = 1/(1 + k)$. The bigger k is, the smaller the rate.

2. Context switch cost is composed of two main costs: direct and indirect overhead. Direct overhead can be measured by getting the cycle count before, then context switching, and then getting the cycle count after: $\text{after} - \text{before} = \text{direct overhead}$. Indirect overhead refers to the time it takes for the hardware to restore/create its optimal state curated for the process we are switching to (e.g. caching).
3. Time per instruction = cycle of the CPU.
4. DRAM is more than $100\times$ slower than a CPU.
5. Temporal locality: if a location in memory has been accessed before it is likely to be accessed again. Spatial locality: if a location in memory has been accessed before, it is likely that nearby locations will also be accessed. That is what caching is for.
6. L1 cache: 4–5 CPU cycles. L2 cache: 12 cycles. L3: 36 cycles. DRAM: 230 cycles.
7. Switching between threads could be cheaper if they are working on the same instructions.
8. The indirect component of the context switch is two times greater than the direct component.

Memory Sharing Between Processes

1. Processes can share memory via opening a shared memory object using `shm_open`, `shmget`, `shm_unlink`. `shmat` attaches the object to the address space of the process, etc.

Terminology

- **Multiprocessing:** when we use multiple cores to run the same job.
- **Multiprogramming:** when we run multiple jobs together (both in the case of a single core and multiple cores).
- **Multitasking:** when we timeslice (same core).

Questions

- What happens when I try to initialize a writer first when there are no readers? What does this even mean?
- Parents do not share the FDT, but if a parent creates a pipe and then forks, the child will have a copy of the FDT with all the open files including the pipe. Makes sense. And when there are multiple threads they just share the FDT with no cloning.
- In the example given in slide 22 of Lecture 3, will the parent get stuck until the child reads? Will a `SIGPIPE` be fired? A `SIGPIPE` will NOT be fired, because there are readers (the child still has not closed the reading end). The parent will NOT get stuck. If, however, the parent tries to write when it also still has the reading end, it WILL get stuck because it is trying to write and will wait for the reading processes to finish before writing. Conversely, if the pipe is for threads then there is no need to close anything.
- Can a previously reading child send a message to its parent?

Tutorial 3

1. In Linux, processes communicate using different means, such as pipes, the filesystem (I/O), sockets, and signals.
2. Signals are numbered from 1 to 31.
3. Signals are implemented using software only.
4. When using `kill()` with `sig=0`, all that does is check if the process with the specified PID actually exists.
5. How does the process know it has a signal? The OS updates the relevant bit in the PCB to signal that a signal of that type needs to be dealt with. Every time the kernel returns from kernel mode to user mode, the OS checks if there are any pending signals, then calls the handler and zeroes the relevant bit. In the case of multiple signals: handled from start to finish.
6. In the PCB we also save the signal actions in a structure that has 31 slots.
7. The handler does not need a context switch but the state of the process is saved before handling the signal nonetheless.
8. When a signal is being handled, all signals of the same type are masked temporarily to avoid reentrancy.
9. In the case of a shell and commands, the process that sends the signals is the shell.
10. How the OS sends signals to processes: the process does something illegal, the CPU or relevant hardware sends an interrupt, the OS handles it and records a signal for the process. Upon returning from kernel mode the handler for the signal is called in user mode.
11. How a process sends a signal to another process: it asks the OS to record a signal, and when the kernel returns to the other process the signal is handled.

Process I/O

1. `stdout` – 1. `stdin` – 0. `stderr` – 2.
2. “Regular” files are on the disk whereas hardware or pipes are in DRAM.
3. Processes have FDTs. FDTs are pointed to by `files.fdt` in the PCB. Every object in the FDT is a file object that has fields like seek pointers, etc. All open files are also stored in the global FDT.
4. `open` is a variadic function. It can receive flags and modes. Returns the new fd upon success and `-1` upon failure. The fd is the lowest fd found viable in the FDT. If we want to create a new file we must pass the `mode` argument.
5. `close` returns 0 upon success and `-1` upon failure.
6. `read` and `write` perform their actions starting from the seek pointer.
7. Upon success, `read` and `write` return the number of bytes. If `count = 0`, `read/write` will do nothing and return 0 immediately.

8. The file object struct contains the following fields: `f_count` (number of pointers to the object – useful for pipes and virtual memory), `f_pos` (the seek pointer), `f_mode` (the file modes), `file_ops` (pointers to the implementation of `read`, `write`, etc.).
9. `execv` does not close any previously opened files and does not mess with the FDT.
10. What is written to a pipe is written to the OS's buffers.
11. `pipe` returns 0 upon success and `-1` upon failure.
12. With pipes there is a reading seek pointer and a writing seek pointer.
13. When attempting to read from an empty pipe and no one is trying to write to it, `read` will return 0. Otherwise, if someone is trying to write, `read` will block.
14. If there is not enough memory to write, the writers will be blocked until the readers read.
15. FIFO is a public file stored in DRAM. It is a file even though it is not on the disk.
16. `mkfifo()`
17. FIFO is a two-sided communication pipe. Has a single fd. Anyone can write or read but must open it first.
18. If a process opens a FIFO for reading it will be blocked until a process tries to write to it.
19. FIFO is not automatically removed. We must remove it using `rm` or `unlink()`.

Questions

- If the process does not use any system calls, how will it get the signal? And why does the OS call the handler – did we not say they are called by the process in user mode?
Answer: it is executed in user mode.
- What happens when a context switch occurs while dealing with an interrupt?

Tutorial 4

1. Modules are added kernel code that we can load at runtime (no need to recompile or reboot the computer). They are used to add new functionality to a program. Character devices and block devices.
2. How a computer is booted: when a PC is turned on, the OS is yet to be in DRAM and needs to be loaded from the disk.
3. How the OS is booted: BIOS is first loaded to memory. It loads the first sector of the MBR. MBR loads the boot loader and the boot loader loads the kernel. The kernel loads `init` and runs it.
4. BIOS identifies hardware devices, checks if the standard I/O works fine, and checks for any bootable devices. If there are none it prints an error; otherwise it loads the first sector.

5. The boot loader allows us to choose which kernel to load. It also loads the initial filesystem `initramfs/initrd`, and the kernel uses the initial filesystem to find `init` and run it.
6. Once `init` is running it uses the info in the initial filesystem to load the actual filesystem and the necessary drivers. From there it is able to load all of the modules and drivers, and it mounts and discards the initial filesystem.
7. Once the actual filesystem is there, the kernel runs `/sbin/init`.
8. Modules are libraries that work in kernel mode.
9. Advantages of using modules: no need for recompilation, no need to reboot, can support newer hardware in the future, kernel code takes up space, and kernel compilation time is lowered.
10. Only root users can load modules.
11. Devices are represented under `/dev/` and are in the filesystem.
12. Drivers work in CPL 0. They map syscalls to specific implementations.
13. Block devices allow for random access and are accessed in units of blocks. Character devices are accessed serially. Linux has a page cache layer that allows for accessing block devices in bytes.
14. `mknod` requires root.
15. `f_ops` in the file object is the driver. Drivers can store data in `private_data`.

Lecture 4

1. Jobs are rectangles in core $\times$ time (core on the y -axis, time on the x -axis). When a process needs multiple cores, it is said to be wide. If it takes a lot of time, it is said to be long.
2. One of the scheduling algorithms is FCFS.
3. Jobs are sometimes tailored to use most of the physical memory of the CPU cores so multiprogramming is not possible. We therefore need to use batch scheduling where cores are dedicated and not shared, and jobs run to completion (non-preemptive).
4. The slowdown factor of a job is the ratio between its response time and its runtime: $1 + w/r$.
5. Utilization: the percentage of time the CPU was utilized.
6. Average wait time is dominated by long jobs, and average slowdown is also dominated by long jobs.
7. Pros of FCFS: fair, easy to implement. Cons: short jobs have to wait a long time, creates fragmentation. Backfilling + FCFS (EASY) is supposed to help prevent a lot of the fragmentation.
8. Pros of backfilling: narrow or short jobs are able to run faster potentially, less fragmentation. Cons: needing to know the runtime in advance and keeping track of all the

fragmentation to be able to patch it. Therefore when users submit jobs they need to have a good estimate of the runtime.

9. SJF – jobs are ordered by their runtime. Optimal for performance reasons. May cause starvation.
10. Convoy effect: when jobs are slowed because a long job is running.
11. Both EASY and SJF suffer from the convoy effect.
12. Given any scheduler S , given that there is one CPU core, that the runtimes of the jobs are known, and that they all arrive simultaneously, then SJF performs better than S in terms of average wait time or just as well.
13. SJBf (Shortest Job Backfilling): like EASY when it comes to scheduling but the backfilling is done in order of shortest to longest wait time.
14. LXF: the wait queue is ordered in terms of largest to smallest slowdown and the backfilling is done in terms of largest slowdown factor too (head of the wait queue). We have to recompute all the values when we finish running and when new jobs arrive since wait time changes.
15. When talking about preemptive schedulers we will ignore I/O or the state “waiting”.
16. We want a scheduler to be preemptive when responsiveness matters to users (e.g. PowerPoint, Word) and when runtimes vary.
17. Quantum: the max amount of time a process is allowed to run before it is preempted. Quanta are more often than not assigned per process, depending on behavior in Solaris and on `nice` in Linux.
18. In preemptive schedulers, response time $\geq$ wait time + runtime (because we need to include context switching and all the quanta cycles). Wait time is the same. Utilization and throughput may take into account context switches.
19. Round robin scheduling – when we have cycled through all the processes in the list, it is called an “epoch”.
20. How does the OS know it is time to preempt? There is an alarm interrupt. It fires every few milliseconds (a constant number). The OS checks if it is time to preempt upon receiving said interrupt.
21. With a huge quantum, RR simulates FCFS.
22. Gang scheduling: RR on multiple cores. Sometimes we want to run jobs that run on several cores, and sometimes we want to run processes together. When we assign a core a process, that process will not run on a different core. When processes terminate, sometimes we can merge time slots and end up with a greater quanta as a result.
23. The price of preemption is very high but will result in better wait and response times.

Lecture 5

1. Assume X is a preemptive scheduler; there exists a batch scheduler S such that $\text{avgResp}(\text{batch}) \leq \text{avgResp}(X)$ (0 context switch overhead).

2. Therefore SJF is also optimal among preemptive schedulers.
3. RR is up to 2 times slower than SJF.
4. SRTF is like SJF but uses preemption, making it optimal.
5. Whenever a job terminates or arrives, SRTF schedules the job with the least remaining time.
6. Selfish RR: new processes are in a FIFO queue and old ones are scheduled using RR. Processes age as well. New processes are scheduled to run when there are no old ones ready to run, or when the newer processes age and become old. Faster aging is RR and slower aging is FCFS.
7. Priority schedulers / general-purpose OS: running more decreases priority and running less increases it. That is why I/O-bound processes get higher priorities.
8. Generally, general-purpose OSes use multi-level priority queues. The queue consists of multiple queues; those at the top have a higher priority and each queue uses RR. Processes move between queues. In some schedulers higher queues are given a shorter quantum.
9. In POSIX each task is associated with one of three scheduling policies: RR, FIFO, and other. RR and FIFO are real-time policies. The default policy is typically a multi-level priority queue with a negative feedback loop.
10. Real-time tasks are favored.
11. When the quantum of all runnable tasks reaches 0, the CPU allocates more time to all tasks, essentially starting a new epoch.
12. Tasks have an integer associated with their priority level. Higher value means higher priority. It is composed of two main numbers: dynamic priority – controlled by the OS (remaining time and current priority) – and static priority, which can be changed by the user using `nice()` or `sched_setscheduler()`. It affects the max quantum for the task.
13. When the task's dynamic priority reaches 0, the CPU is yielded.
14. The OS gets a timer interrupt every $1/\text{HZ}$ seconds = 10 milliseconds; typically $\text{HZ} = 100$.
15. Dynamic priority's units are ticks.
16. Every `task_struct` has 5 fields: `counter`, `mm`, `processor`, `nice`, `need_resched`.
17. Kernel's nice is between 1 and 40, is 20 by default, and can be changed using `nice` and `sched_setscheduler`.
18. User's nice is between -20 and 19.
19. $\text{Kernel nice} = \text{user nice} + 20$.
20. `goodness(task, cpu)` determines how desirable a task is to a CPU.
21. `reschedule_task` reschedules, preferably on an idle core, but preempts if there is none.
22. I/O-bound tasks by default have a tick counter of 12 ticks.

23. If the task is I/O-bound it converges to 2α .

Tutorial 5

1. How does SRTF work? When a new job arrives, the algorithm checks if the time it needs or any of the other jobs in the queue is less than the time the current running job needs, and if that is the case it preempts it. SRTF is optimal when context switch time is 0.
2. We can modify SRTF such that it is also applicable for I/O processes. That can be achieved by cutting up the processes that need I/O into segments.
3. I/O-bound processes are more interested in wait times. CPU-bound processes are more interested in response times.
4. SJF and SRTF are not good when it comes to wait time.
5. Quanta must be a multiple of $1/\text{HZ}$.
6. Only root users can define processes as real-time.
7. Real-time processes need a quick response time whereas regular processes do not.
8. If there are real-time processes in the system ready to run, the algorithm chosen is the scheduling algorithm for real-time processes; otherwise, it is the other one.
9. The algorithm for scheduling processes: there is a multi-level queue. The higher a task is the lower its priority. Each task has a scheduling policy set by `sched_setscheduler`.
10. When the policy is FIFO, the process will never yield the CPU unless: there is a higher-priority real-time process waiting to run, it calls the syscall `yield`, or it is using I/O.
11. New processes or processes that have been unblocked will be sent to the back of the line of their respective priority level.
12. Timeslice: $100 \times \text{HZ}/1000$.
13. CFS scheduler: achieves both efficiency when making a decision about who to run next and scalability.
14. It is fair in the sense that it gives all processes an equal amount of time (up to priority level).
15. Virtual runtime: when a process runs it accumulates virtual runtime. CFS will choose the process with the lowest virtual runtime. It keeps track of the max and min virtual runtimes.
16. The algorithm will dedicate `sched_latency/N` time to each process.
17. CFS is like RR but with a dynamic quantum time.
18. Once a process depletes its quantum, CFS chooses the process with the lowest virtual runtime.
19. There is a minimum quantum time of 6 ms.

20. CFS keeps a red-black tree ordered according to virtual runtime. When new processes are added they are inserted with the max virtual runtime.
21. When a process sleeps or is blocked it is removed from the tree, and when it wakes up it is re-added with the min virtual runtime.
22. CFS also allows for an uneven distribution between processes using `nice`. The nicer the process is, the lower its priority.
23. $vr += \frac{w_0}{w_i} \cdot \Delta t$
24. $Q_i = \frac{w_i}{\sum_j w_j} \cdot \text{sched_latency}$

Lecture 6

1. When the outcome of a program is not deterministic due to the timing of certain events, we say that there exists a race condition.
2. Shared (global) variables/structures (in the case of threads) need to be monitored/fenced/protected.
3. Atomic operations / critical section – ensures no partial results. Critical sections do not have to be the same code across all threads.

Tutorial 6

1. Cons of context switch: bad utilization.
2. When we want to preempt a program, it is done because an alarm interrupt has been fired. This is handled in the OS, and so as part of handling the alarm interrupt we can also context switch while we are in the kernel.
3. `schedule()` is the only way to context switch in Linux.
4. The only way to enter the OS is through interrupts or syscalls.
5. Context switching does not require loading certain memory address spaces or anything, only changing the ones we are working with. Using `cr3` – that is where the process's memory is.
6. Where is the process's context? The stack, heap, PCB, and all are already in the memory and as per point 5 we would not need to reload them. We do, however, need to back up and restore the registers. The backing up and restoration is done in both the kernel stack and the `thread` field in the PCB. Also the base of the process's kernel stack in `TSS.sp0`.
7. When a process is interrupted via an interrupt, the context is saved on the kernel stack. The kernel stack of the current process is saved in `TSS.sp0`.
8. Registers are per-CPU.
9. TSS is per-CPU, pointed to by the TR register, and is saved in kernel memory.
10. In the `thread` struct in the PCB there exists a field that points to the top of the kernel stack of the process, called `rsp`.

11. What we actually want to do is switch between the kernel stacks of the processes.
12. Context switch phases: `__switch_to_asm` (for registers) → `switch_to` (for TSS). In `switch_to`: first we switch the `mm` and then we switch registers. `__switch_to_asm` has to save the registers (ABI) and then swap the previous `rsp` with the next `rsp` in the `thread` field.
13. By convention, the `rsp` of next is going to point at the saved registers (next has undergone a context switch in the past so its stack will look similar: registers → return address → context switch → schedule).
14. Then using the new `rsp`, `__switch_to_asm` restores next's registers and jumps to `switch_to`, which restores `TSS.sp0` and then returns.
15. In the case of the first ever process: `do_fork` simulates a context switch's behavior. Saves the registers on its stack like `__switch_to_asm` would, using the structs `pt_regs` and `inactive_task_frame`.
16. What `do_exit` does: releases all of the memory and files, etc. Saves exit code in `exit_code` in the PCB. Updates family relations and assigns children to init. Changes the task's status to zombie. Calls `schedule()`.
17. `wait` actually deletes the process altogether using `release_task`.

Lecture 7

1. Locks are implemented using hardware.
2. Spinlocks: will spin until they acquire the lock or until they are preempted (`while` loop).
3. When we are trying to acquire a spinlock, we disable interrupts.
4. Do we need to disable interrupts? In a uni-core system: yes, in case the handlers use synchronized structures. In a multi-core system: no.
5. Spinlocks: only use if we are spinning for a short period of time and the context switch time is greater.
6. Semaphores do not have to be released by the locking thread.

Tutorial 7

1. Because threads are essentially processes, they have PCBs and PIDs. The PID of the main thread can be accessed using `getpid()`. It is stored in the PCB (`tgid`). In the PCB there is also a list of all the threads. Use `gettid()` to get the thread ID.
2. We use `clone` to create threads.
3. C statements are oftentimes not atomic, so a context switch can happen in the middle of one.

Questions

- Why does `kill` impact everything? If the process receives a signal, who deals with it?

Lecture 8

1. Networking consists of 5 layers. Layer 5 (top): SSH and HTTP. Layer 4: TCP/UDP. Layer 3: IP. Layer 2: Ethernet. Layer 1: hardware. Layers 2–4 are in the kernel; Layer 5 is user space; Layer 1 is hardware.
2. L1: hardware. L2: data link. L3: network. L4: transport. L5: application.
3. L5 is determined by the application.
4. What is sent is not always fully received, mostly because it is too big for the buffers in networks.
5. Networks might mess up the order of messages.
6. L4 is typically implemented by the kernel (TCP, UDP, ...).
7. TCP is used because it cares about loss and reordering. It ensures that all bytes arrive in the correct order. Needs to establish a connection.
8. TCP splits bytes into segments. When a segment is received it needs to be acknowledged. Segments that have not been acknowledged are resent. Transmission is slowed down by the receiver when a lot of segments are missing, and gradually speeds up (congestion/flow control). The sender is forbidden to send more than what the buffer can hold.
9. There are congestion control algorithms: BBR, Cubic, Reno. BBR (by Google) has sharper drops but they are less frequent.
10. Cubic has more frequent drops but they are less sharp.
11. Reno has both frequent and sharp drops.
12. UDP does not care – the opposite of TCP.
13. UDP uses datagrams or chunks. Per-chunk integrity. Simpler, low latency, no flow control, no need to establish a connection.
14. Domain names/hostnames are translated to IP addresses. IPv4: 32-bit address. IPv6: 128-bit address.
15. DNS helps translate domain names to IP addresses.
16. Each IP is associated with one host, whereas a host can have multiple IPs.
17. IP is implemented by the IP protocol.
18. Parties communicate through sockets, which are file descriptors created using `socket()`. Functions: `send()`, `recv()`, `sendmsg()`, `recvmsg()`.
19. Hosts can have multiple communicating processes that use different sockets, and that is what ports are for. Ports are 16-bit unsigned integers. Each chunk is associated with a port and an IP.
20. Ports below 1024 are reserved.

21. Socket unique identifier: 5-tuple (source IP, source port, dest IP, dest port, TCP/UDP).
22. `tcp_establish` – server; `tcp_connect` – client. Both need to use `getaddrinfo` and invoke `socket()` using the `addrinfo`. `getaddrinfo(host, client, input, output)`.
23. Local area network (LAN): devices are connected, each device has a NIC which is connected to a switch. LAN devices are nodes. They use Ethernet as a protocol to communicate with each other.
24. Ethernet is both an L1 and L2 protocol. Byte chunks in L2 are called frames.
25. Ethernet does not need a connection and can be lossy.
26. There is no connection between Ethernet and TCP/UDP.

Lecture 9

1. The OS caches disk content in RAM. When a process writes to disk content that is cached in RAM, the OS writes it to RAM and it is later synced to the disk.
2. This area that caches disk data is called the page cache.
3. Pages and frames have the same size 2^k .
4. Typically $k = 12$, therefore $|\text{page}| = 4 \text{ KB}$.
5. Pages can reside in memory, on disk, or be unallocated entirely.
6. For each virtual address, we divide it into two parts: the page and the offset. The page is what gets translated to a physical frame and the offset is just the offset. This results in a PA of $[\text{frame}][\text{offset}]$.
7. Each process has a page address table. It has bits and the frame that the page is translated to, with $2^{32}/2^{12} = 2^{20}$ entries in the case of $k = 12$.
8. Upon each access to memory, the access mode should be checked.
9. The OS will produce a major page fault in the case that a page residing on disk is accessed.
10. The CPU discovers major page faults (valid bit = 0) and cannot do anything about it on its own, so it fires an interrupt to the OS. The OS then executes the handler for the fault, suspends the faulting process, and context switches. When handling is complete, the suspended process becomes runnable and the operation is restarted.
11. With minor page faults, the process remains runnable and there is no need to access the drive (COW, lazy memory allocation, reading a file already in the page cache).
12. `mmap`: maps a file to the address space of a process. The file becomes an array of bytes. Using the write flags makes writing/reading to/from the array simulate writing/reading to/from a file. It is durable on disk “soon” (in batches). Can use the `sync` function.
13. `mmap` makes the relevant array pages point to the pages in the page cache. All I/O operations go through the page cache.
14. Pros of using it: saves overhead cost of constantly using `write` or `read`. Zero-copy.

15. Anonymous pages are allocated to the heap or stack. They are not file pages.
16. `MAP_SHARED` – affects the file. `MAP_PRIVATE` – does not affect the file.
17. Paging or swapping: when we spill a page onto disk. Swap space is a space on disk for swapped pages. Only holds anonymous pages.
18. On-demand paging: pages start out on disk and are swapped in if they are being accessed for the first time. Also works for anonymous memory mapping.
19. $ws_p(k)$: the pages accessed by p in the last k accesses. These are called the working set.
20. Thrashing is when we overcommit too much memory and the system will just be handling faults with nothing getting done.
21. Readahead: swapping in additional pages that have not actually been demanded.
22. TLB caches recent VA-to-PA translations.
23. Processes do not know that the TLB exists.
24. Hardware fills the TLB.
25. The OS can invalidate entries in the TLB.
26. Victim pages are the pages that need to be swapped out.
27. The process of swapping out pages is typically done using a daemon. When the number of free pages surpasses a certain threshold it slows down, otherwise it speeds up.
28. The clock algorithm for swapping out pages: a bit says “was this accessed since the last time I checked”. Periodically (every n seconds) that bit is zeroed. We keep a pointer to the page we last checked. The first page for which the bit value is 0 is the victim page. As we iterate we zero the bits that are 1. (LRU approximation.)
29. NRU is like LRU but prefers keeping dirty pages. It adds another bit for whether the page is dirty.

Questions

- Do frames and pages need to have the same size?

Tutorial 9

1. UDP provides a basic protection: if the data arrives it is most likely complete.
2. Use UDP when latency is what matters.

Tutorial 10

1. Cons of using physical addresses: fragmentation and security.
2. External fragmentation is when there is enough space in memory collectively but it is scattered and does not serve us. Internal fragmentation is the overhead of managing the allocation and its metadata and/or alignment.

3. Demand paging: only actually allocating memory when it is about to be used.
4. Deduplication: when two or more processes are reading the same data.
5. Copy-on-write: prevents over-copying.
6. Paging in 32-bit: $|\text{frame}| = |\text{page}| = 4 \text{ KB} = 2^{12}$. There are $2^{32}/2^{12} = 2^{20}$ pages in virtual memory (and frames in physical memory) = 1 M.
7. Each process has a page table that translates page addresses to frame addresses.
8. In the page table we also mark the access mode of the page.
9. Hierarchical page table: to know how many pages there are in each block we divide the frame size by the page entry size. We need to know how many pages there are in each block to determine the size of the top layer of the page global directory. Higher levels in the tree mean division, not modulo.
10. The hierarchical page table is called a page global directory (PGD). Entries in it are called PDEs.
11. `cr3` points to the page table.
12. If `present` = 1 the page is in memory. If `present` = 0 the page is either unallocated or is in the swap space on disk.
13. The CPU while translating will look in the TLB first and then will attempt to translate.
14. The OS has to invalidate addresses in the TLB when needed and when a context switch happens.
15. When should the OS not invalidate the TLB? When the thread to run next has the same `mm` as the currently running one, and/or if the thread about to run is a kernel thread. The kernel does not have a page table of its own and so uses the page table of the currently running process. Moreover, threads work on the kernel stack.

Lecture 11

1. In the entries of the PGD there exists the global bit: it helps the OS identify which of these are OS pages so it can keep them in the TLB.
2. Hardware vs OS: hardware is responsible for the structures, amount of bits, etc., and the OS is what fills those bits. In addition, hardware does the page walk and sets the accessed and dirty bits; the OS turns these off.
3. Syncing and invalidating TLBs – OS. Populating TLBs – hardware.

Tutorial 11

(No notes.)

Lecture 12

1. We want hard drives to be robust and reliable.

2. We need an abstraction, and that is what the filesystem is for.
3. Writes to files should be atomic to readers (either everything or nothing at all).
4. Hard links vs soft links: when creating a hard link a pointer to the file is created, and soft links create a new file that has the path to the file they are linked to.
5. Files have reference counts, and when a reference count reaches 0 the file should be removed. But if there are processes that have file descriptors referencing the file, the contents are not really removed until all references are removed as well.
6. Hard links to directories are not allowed.
7. `.` and `..` are hard links.
8. Soft links can be broken, are not counted as part of the reference count, and can point to files outside the system and to directories as well.

Tutorial 13

1. The name of the file is not kept in the inode – only in the directory in which the file exists.